Simulador de Arreglos RAID 0, 1, 5, 10

Michelle Sayas y Brandon Zanotti

September 2026

1 El Problema

El almacenamiento masivo de datos en sistemas de cómputo modernos enfrenta un desafío fundamental de diseño: el equilibrio entre rendimiento de entrada/salida (E/S), capacidad de almacenamiento útil y tolerancia a fallos ante averías físicas de hardware. Las unidades de almacenamiento de disco individual representan puntos únicos de fallo que limitan la velocidad de acceso y comprometen la disponibilidad de los datos.

Para abordar estas limitaciones, se introdujo la tecnología RAID (Redundant Array of Independent Disks), la cual combina múltiples discos físicos en una o más unidades lógicas gestionadas de forma coherente. El problema planteado consiste en diseñar y modelar un simulador en C++ que abstraiga el funcionamiento atómico de la memoria secundaria y permita evaluar de manera interactiva el comportamiento de cuatro arquitecturas principales de RAID (0, 1, 5 y 10):

- **RAID 0 (Striping / Distribución por bloques):** Busca maximizar el rendimiento dividiendo la información secuencialmente entre todos los discos. El problema crítico de esta arquitectura radica en que carece por completo de redundancia; si un solo disco se avería, la totalidad del arreglo queda inoperativa y los datos se pierden de forma irreversible.
- **RAID 1 (Mirroring / Espejeo):** Prioriza la alta disponibilidad manteniendo una copia exacta (1:1) de la información en dos unidades físicas. Su problema principal radica en la ineficiencia espacial, ya que penaliza la capacidad útil al 50% del total de almacenamiento instalado.
- **RAID 5 (Paridad Distribuida):** Resuelve la ineficiencia del espejeo distribuyendo bloques de datos junto con bloques de control de paridad a lo largo de mínimo 3 discos. Permite soportar la caída de 1 disco sacrificando únicamente la capacidad de una unidad ($N - 1$). El problema computacional se centra en el cálculo dinámico de la paridad mediante operaciones lógicas a nivel de bits y en la reconstrucción "al vuelo" del bloque perdido durante lecturas degradadas.

- **RAID 10 (Arreglo Híbrido - Striping + Mirroring):** Combina la distribución del RAID 0 con la replicación en parejas del RAID 1 para ofrecer alto rendimiento y tolerancia a múltiples fallos. Su limitación se halla en la exigencia de un número par de discos (mínimo 4) y la ineficiencia espacial del 50%, garantizando tolerancia a fallos siempre y cuando no caigan simultáneamente los dos integrantes de una misma pareja de espejos.

El simulador debe integrar estas reglas de negocio en una arquitectura modular orientada a objetos en C++, permitiendo inyectar fallos controlados, reconstruir discos degradados y auditar visualmente el mapa de memoria resultante.

2 Técnicas Computacionales para su Solución

Para solucionar el problema planteado, el sistema implementa una arquitectura orientada a objetos fundamentada en encapsulamiento, herencia, polimorfismo y estructuras de datos dinámicas.

2.1 Estructuras de Datos Principales

2.1.1 Estructura `Bloque` (`bloque.hpp`):

- Representa la unidad contenedora atómica dentro del simulador.
- Almacena una variable de tipo `std::string contenido` para los caracteres de datos o información de redundancia.
- Define una bandera booleana `esParidad` (inicializada en `false`) para diferenciar semánticamente la información de usuario de los bloques de paridad.

2.1.2 Clase `Disco` (`disco.hpp`)

- Modela el comportamiento y estado operativo de un disco duro físico individual.
- Encapsula un identificador entero `int id`, un estado booleano `bool activo` y un contenedor vectorial `std::vector<Bloque> bloques` que actúa como la memoria secundaria simulada.
- Incorpora el método `fallar()`, el cual invalida el disco (`activo = false`) y destruye el contenido de la memoria ejecutando `bloques.clear()`.
- Define el método `reemplazarYReconstruir(size_t numBloques)`, que restablece la unidad (`activo = true`) y reinicializa el vector con bloques vacíos preparados para recibir la reconstrucción.

2.1.3 Clase Abstracta Base RAIDSystem (raidsystem.hpp):

- Define el contrato polimórfico común para todos los niveles RAID.
- Almacena las unidades en un contenedor protegido `std::vector<Disco> discos` e inicializa dinámicamente los objetos `Disco` en su constructor mediante `emplace_back(i)`.
- Declara la interfaz de métodos virtuales puros: `escribirCadena(), leerCadena(), reconstruirDisco() y mostrarEstado()`.
- Implementa el método concreto `simularFalloDisco(int discoId)`, validando los rangos de índices antes de invocar la caída del disco.

2.2 Patrones de Diseño y Gestión de Memoria

- Polimorfismo y Punteros Inteligentes (`proyecto.cpp`): El controlador principal gestiona la instancia del simulador a través de un puntero inteligente `std::unique_ptr<RAIDSystem> raid`. Esto permite instanciar dinámicamente mediante `std::make_unique` clases derivadas (`RAID0, RAID1, RAID5 o RAID10`) según la selección del usuario en tiempo de ejecución, garantizando la liberación automática de memoria sin fugas (memory leaks).

2.3 Algoritmos Computacionales por Nivel RAID

2.3.1 Algoritmo de RAID 0 (Striping - `raid0.hpp`):

- **Escritura:** Recorre la cadena de datos y asigna cada carácter al disco destino utilizando la operación módulo: $discoTarget = i \pmod{numDiscos}$.
- **Lectura:** Reconstruye la cadena concatenando los bloques fila por fila y columna por columna. Si detecta que cualquier disco tiene `activo == false`, interrumpe el proceso emitiendo una alerta de falla catastrófica.
- **Reconstrucción:** Retorna de inmediato `false`, dado que carece de información redundante.

2.3.2 Algoritmo de RAID 1 (Mirroring - `raid1.hpp`):

- Restricción: Fija la arquitectura a exactamente 2 discos (`RAIDSystem(2)`).
- **Escritura:** Por cada carácter ingresado, inserta una copia idéntica de manera simultánea en `discos[0] y discos[1]`.
- **Lectura:** Identifica el primer disco operativo (`discoValido`). Si ambos están caídos retorna error; de lo contrario lee secuencialmente de la copia intacta.
- **Reconstrucción:** Si un disco cae, el algoritmo valida que el espejo permanezca activo, reinicializa el disco dañado y copia íntegramente el vector de bloques del disco funcional.

2.3.3 Algoritmo de RAID 5 (Paridad Distribuida y XOR - `raid5.hpp`):

- **Lógica de Paridad en Bits:** Implementa la función privada `calcularXOR()`, aprovechando la propiedad idempotente de la compuerta XOR ($A \oplus B = P \implies A = P \oplus B$). La paridad de un conjunto de caracteres se calcula acumulando operadores: `char res = 0; for (char c : caracteres) res ^= c;`.
- **Rotación Inversa de Paridad:** Agrupa la cadena en franjas o stripes de tamaño ($numDiscos - 1$). En cada franja, el disco encargado de almacenar la paridad rota secuencialmente mediante la fórmula:

$$parityDisk = (numDiscos - 1) - (stripe \pmod{numDiscos})$$

- **Lectura Degradada:** Si un disco de datos cae, la lectura no se interrumpe. Por cada franja, se agrupan los caracteres de los discos funcionales (incluyendo el bloque de paridad) y se ejecutan en `calcularXOR()`, regenerando instantáneamente el carácter faltante.
- **Reconstrucción:** Valida que el número de discos inactivos no sea superior a 1. Recorre cada franja y recalcula el contenido del bloque extinto aplicando la compuerta XOR sobre el resto de las unidades activas.

2.3.4 Algoritmo de RAID 10 (Striping de Parejas Espejo - `raid10.hpp`):

- **Restricción:** Exige una cantidad par de discos mayor o igual a 4.
- **Escritura Híbrida:** Divide los discos en $numParejas = numDiscos/2$. El carácter i de los datos se asigna a la pareja:

$$parejaTarget = i \pmod{numParejas}$$

  Posteriormente, escribe el bloque en el disco primario de la pareja ($disco1 = parejaTarget2$) y en su réplica ($disco2 = disco1 + 1$).
- **Lectura y Tolerancia:** Evalúa la integridad de cada pareja individualmente. Si una pareja pierde sus 2 discos (`!discos[p*2].activo && !discos[p*2+1].activo`), el arreglo cae. Si al menos un integrante de cada pareja sigue activo, extrae los datos alternados leyendo del disco sobreviviente.
- **Reconstrucción:** Identifica el espejo asociado mediante la expresión `(discoId % 2 == 0) ? discoId + 1 : discoId - 1` y restaura los bloques mediante copia directa.

3 Análisis y Comparación de Resultados

A partir del desarrollo y pruebas en el simulador interactivo (`proyecto.cpp`), se obtuvieron datos empíricos sobre la eficiencia computacional, capacidad útil y resiliencia de cada nivel de RAID frente a inyecciones de fallos.

3.1 Análisis de Capacidad Útil vs. Tolerancia a Fallos

- **RAID 0:** Ofrece una capacidad útil del 100% de la memoria física agregada (NC). Sin embargo, la inyección de un único fallo destruye la cadena y devuelve error en la operación de lectura, confirmando cero tolerancia a fallos.
- **RAID 1:** Proporciona únicamente un 50% de capacidad útil ($1C$), consumiendo la mitad del espacio en la réplica exacta. Soporta la falla de 1 de los 2 discos sin pérdida de información, restaurando el 100% del estado mediante copia limpia.
- RAID 5: Alcanza un rendimiento de capacidad de $\frac{N-1}{N}100\%$, representando un aprovechamiento óptimo a medida que aumenta el número de discos (ej. 66.6% en 3 discos, 75% en 4 discos). Tolera la pérdida de hasta 1 disco simultáneo. Si caen 2 discos, el algoritmo bloquea la reconstrucción y la lectura degradada, devolviendo un error.
- **RAID 10:** Presenta una capacidad útil fija del $50\%(\frac{N}{2}C)$. Su ventaja clave radica en una tolerancia a fallos descentralizada: en un arreglo de 4 discos, puede soportar la caída de hasta 2 discos siempre que pertenezcan a parejas distintas (ej. Disco 0 y Disco 2). Si la falla afecta a ambos integrantes de una misma pareja (ej. Disco 0 y Disco 1), la lectura reporta la caída catastrófica del sistema.

3.2 Tabla Comparativa de Rendimiento y Arquitectura

Métrica / Nivel RAID	**RAID 0**	**RAID 1**	**RAID 5**	**RAID 10**
Capacidad util	$100\%(NC)$	$50\%(1C)$	$\frac{N-1}{N}100\%$	$50\%(\frac{N}{2}C)$
Discos mínimos	2 unidades	2 unidades	3 unidades	4 unidades (pares)
Tolerancia máxima a fallos	0 discos	1 disco	1 disco	Hasta 1 disco por parejas
Mecanismo redundante	Ninguno	Espejeo Directo (1:1)	Cálculo Paridad XOR ($\oplus$)	Espejeo en Parejas Distribuidas
Complejidad de reconstrucción	No soportado ($O(1)$)	Copia Directa ($O(M)$)	Recuperación por Cómputo XOR ($O(NM)$)	Copia Directa de Espejo ($O(M)$)
Impacto de lectura degradada	Error Inmediato	Nulo (Lee del espejo)	Medio (Cálculo XOR al vuelo)	Nulo (Lee del espejo activo)

(Donde N es la cantidad de discos del arreglo y M la cantidad de bloques almacenados por disco).

3.3 Conclusiones del Análisis

- **Balance Costo/Beneficio:** El nivel RAID 5 demostró ser la solución matemáticamente más refinada para optimizar recursos. Su algoritmo de paridad redujo la pérdida de espacio útil a solo un disco, a cambio de una carga computacional en operaciones de bits (XOR) durante lecturas degradadas y reconstrucciones.

- **Redundancia Crítica vs. Desempeño:** RAID 10 demostró ser superior a RAID 1 en escalabilidad y superior a RAID 5 en simplicidad de recuperación (copia directa sin cálculo al vuelo), consolidándose como la mejor opción para aplicaciones críticas que requieren alto rendimiento de lectura/escritura y alta disponibilidad.

- **Validación del Software:** Las abstracciones creadas en C++ (`Bloque`, `Disco y RAIDSystem`) demostraron representar fielmente los axiomas de la virtualización de almacenamiento a nivel de hardware, permitiendo auditar y verificar el comportamiento determinista de cada algoritmo ante la inyección de errores.